الاحتراف الكامل

في مكتبات بايثون لعلم البيانات والتعلّم الآلي

الجزء الأول: مكتبة NumPy

الفصول ١ - ٦: من التعريف بالمكتبة حتى الدوال الأساسية

مرجع احترافي للمبرمجين والمهندسين وعلماء البيانات

جدول المحتويات

جدول المحتويات 2
مقدمة الكتاب 4
الفصل الأول: التعريف بمكتبة NumPy 5
١.١ ما هي NumPy ولماذا ظهرت 5
١.٢ نبذة تاريخية 5
١.٣ أهم الاستخدامات 5
١.٤ شركات ومشاريع تعتمد عليها 5
١.٥ المميزات 6
١.٦ العيوب والحدود 6
١.٧ متى تستخدمها ومتى لا تستخدمها 6
١.٨ أشهر البدائل 7
ملخص الفصل 7
الأسئلة الشائعة 7
الفصل الثاني: التثبيت والإعداد 8
٢.١ طرق التثبيت 8
٢.٢ البيئات الافتراضية 8
٢.٣ إدارة الإصدارات والتوافق 8
٢.٤ أفضل أسلوب للاستيراد 8
أفضل الممارسات 9
٢.٥ تنظيم المشروع والملفات 9
ملخص الفصل 9
الفصل الثالث: فلسفة التصميم (Design Philosophy) 10
٣.١ الفكرة الجوهرية: مصفوفة واحدة متجانسة 10
٣.٢ فصل الوصف عن البيانات: Metadata وBuffer 10
٣.٣ Vectorization كفلسفة لا كميزة إضافية 10
٣.٤ Broadcasting: تعميم العمليات بدون نسخ 10
٣.٥ البنية الداخلية: Modules وNamespaces 11
٣.٦ فلسفة التوافق العكسي وسياسة NEP 11
ملخص الفصل 11
الفصل الرابع: المفاهيم الأساسية (Core Concepts) 12
٤.١ ndarray: الكائن المركزي 12
٤.٢ الأبعاد والشكل (Dimensions & Shape) 12
٤.٣ Stride: كيف تُقرأ الذاكرة فعليًا 12

٤.٤ Views مقابل Copies 12
٤.٥ Broadcasting بالتفصيل 13
٤.٦ الفهرسة والتقطيع (Indexing & Slicing) 13
الفهرسة الأساسية 13
الفهرسة المتقدمة (Fancy Indexing) 13
الفهرسة المنطقية (Boolean Indexing) 13
٤.٧ إعادة التشكيل والنقل والتفريد 14
الأخطاء الشائعة 14
تمارين عملية 14
حلول مقترحة (تلميحات) 14
الفصل الخامس: الكائنات وواجهات البرمجة (Objects & APIs) 15
٥.١ الكائن ndarray: الخصائص الأساسية 15
٥.٢ أنواع البيانات (dtype) 15
٥.٣ الدوال مقابل الطرق (Functions vs Methods) 15
٥.٤ نموذج التنفيذ: Eager Execution 15
٥.٥ Vectorization كنموذج تنفيذ 16
٥.٦ نموذج الذاكرة (Memory Model) 16
أفضل الممارسات 16
الفصل السادس: أهم الدوال (Core Functions) 17
٦.١ دوال إنشاء المصفوفات 17
٦.٢ العمليات الحسابية والإحصائية 17
٦.٣ الجبر الخطي (numpy.linalg) 18
٦.٤ الوحدة العشوائية (numpy.random) 18
٦.٥ ملخص Edge Cases شائعة 18
الأخطاء الشائعة 19
الأسئلة الشائعة 19
تمارين عملية 19
حلول مقترحة (تلميحات) 19
مراجع وروابط رسمية 19
خاتمة الجزء الأول 20

مقدمة الكتاب

هذا الكتاب ليس مذكرة لتعلّم مبادئ التعلّم الآلي أو الرياضيات الكامنة خلفه، بل هو مرجع احترافي متعمّق في المكتبات البرمجية التي يقوم عليها عالم Data Science وMachine Learning في بايثون. الهدف هو الوصول إلى مستوى احترافي حقيقي في التعامل مع هذه المكتبات: قراءة التوثيق الرسمي بسهولة، فهم المشاريع مفتوحة المصدر التي تعتمد عليها، وكتابة كود نظيف يعكس فهمًا عميقًا لفلسفة كل مكتبة وليس مجرد حفظ للدوال.

يبدأ هذا الجزء الأول بمكتبة NumPy، لأنها حجر الأساس الذي تُبنى عليه كل المكتبات الأخرى تقريبًا: Pandas مبنية فوق مصفوفات NumPy، وMatplotlib وPlotly يرسمان بيانات NumPy، وScikit-learn يتوقع أن تكون بياناته على هيئة مصفوفات NumPy. لذلك فإن إتقان NumPy بعمق هو استثمار مباشر في إتقان كل ما بعده.

يغطي هذا الجزء الفصول من الأول إلى السادس: التعريف بالمكتبة وتاريخها، التثبيت والإعداد، فلسفة التصميم، المفاهيم الأساسية، الكائنات وواجهات البرمجة، ثم أهم الدوال. أما الفصول من السابع إلى الحادي عشر (الأمثلة العملية، الأداء، التصحيح، أفضل الممارسات، والمشاريع المصغّرة) فستأتي في الجزء الثاني.

الفصل الأول: التعريف بمكتبة NumPy

١.١ ما هي NumPy ولماذا ظهرت

NumPy، اختصار لعبارة Numerical Python، هي المكتبة الأساسية للحوسبة العددية في بايثون. قبل ظهورها، كانت بايثون لغة عامة الأغراض ممتازة في القراءة والكتابة، لكنها كانت بطيئة جدًا في العمليات الحسابية الكثيفة مقارنة بلغات مثل C وFortran، بسبب طبيعتها المفسَّرة (Interpreted) ونظام الكائنات الديناميكي فيها الذي يجعل كل عنصر في القائمة كائنًا Python مستقلاً له غلاف (Overhead) في الذاكرة والمعالجة.

المشكلة التي جاءت NumPy لحلّها هي: كيف نحصل على سرعة لغات منخفضة المستوى مثل C، مع الاحتفاظ بسهولة وبساطة بايثون؟ الحل كان بناء بنية بيانات جديدة تمامًا اسمها ndarray (N-dimensional array)، وهي مصفوفة متجانسة العناصر (كل العناصر من نفس النوع) مخزَّنة في كتلة واحدة متصلة من الذاكرة، بحيث تُنفَّذ العمليات عليها عبر شيفرة C مُجمَّعة (Compiled) مسبقًا بدلاً من حلقات بايثون البطيئة.

بهذا الشكل، عندما تجمع مصفوفتين من مليون عنصر باستخدام NumPy، فإن الحلقة الفعلية التي تدور فوق العناصر لا تُنفَّذ في مفسّر بايثون، بل في شيفرة C محسَّنة، وهو ما يُعرف بمبدأ Vectorization، وسيكون هذا المبدأ محور الفصول القادمة بأكملها.

١.٢ نبذة تاريخية

تعود جذور NumPy إلى أواخر التسعينيات مع مكتبة Numeric التي طوّرها Jim Hugunin، والتي كانت أول محاولة جادة لإضافة دعم مصفوفات عددية سريعة إلى بايثون. لاحقًا ظهرت مكتبة منافسة اسمها Numarray قدّمت تحسينات لكنها لم تكن متوافقة تمامًا مع Numeric، مما قسّم المجتمع العلمي بين مكتبتين غير متوافقتين.

في عام ٢٠٠٥، قام Travis Oliphant بدمج أفضل ما في المكتبتين وأعاد كتابة الأساس ليُخرج مكتبة NumPy كما نعرفها اليوم، بهدف توحيد الجهود وإنشاء معيار واحد للحوسبة العددية في بايثون. منذ ذلك الحين أصبحت NumPy مشروعًا مفتوح المصدر يقوده مجتمع علمي واسع، وتُعد اليوم من أكثر مشاريع بايثون العلمية استقرارًا ونضجًا، مع إصدارات منتظمة وتوثيق رسمي شامل على numpy.org.

١.٣ أهم الاستخدامات

- الحوسبة العلمية والهندسية: محاكاة فيزيائية، معالجة إشارات، جبر خطي.
- الأساس الحسابي لمكتبات علم البيانات: SciPy، Scikit-learn، Pandas كلها مبنية فوق NumPy.
- معالجة الصور والصوت: الصورة الرقمية هي في جوهرها مصفوفة NumPy ثلاثية الأبعاد (ارتفاع × عرض × قنوات لونية).
- التعلّم العميق: مكتبات مثل TensorFlow وPyTorch تتبنى واجهة برمجية مشابهة جدًا لواجهة NumPy (يُطلق عليها أحيانًا NumPy API Standard).
- التحليل الإحصائي والعمليات المصفوفية الكبيرة في التمويل والاقتصاد القياسي.

١.٤ شركات ومشاريع تعتمد عليها

تعتمد على NumPy عمليًا كل الشركات التي لديها فريق علم بيانات أو تعلّم آلي بلغة بايثون، من شركات التقنية الكبرى إلى المؤسسات المالية والبحثية. من أشهر المشاريع مفتوحة المصدر التي تعتمد عليها بشكل مباشر: Pandas، Scikit-learn، SciPy، Matplotlib، TensorFlow، PyTorch، Astropy (في الفلك)، وOpenCV في بايثون (لمعالجة الصور). ورقة NumPy البحثية المنشورة في مجلة Nature عام ٢٠٢٠ تُعد من أكثر الأوراق استشهادًا في تاريخ البرمجيات العلمية، وهو مؤشر واضح على مكانتها المحورية.

١.٥ المميزات

الميزة	الشرح
الأداء	عمليات مُنفَّذة في C، أسرع بعشرات إلى مئات المرات من حلقات بايثون العادية.
كفاءة الذاكرة	تخزين متجانس ومتصل للعناصر بدل كائنات بايثون منفصلة لكل عنصر.
Vectorization	كتابة عمليات على مصفوفات كاملة دون حلقات صريحة.
Broadcasting	تنفيذ عمليات بين مصفوفات بأشكال مختلفة دون نسخ بيانات غير ضرورية.
التكامل	أساس مشترك تعتمد عليه معظم مكتبات علم البيانات والتعلّم الآلي في بايثون.
النضج والاستقرار	مشروع قديم نسبيًا، موثّق جيدًا، ومُختبر على نطاق واسع.

١.٦ العيوب والحدود

- لا تدعم موازاة (Parallelism) عبر عدة أجهزة أو GPU بشكل مدمج؛ لذلك ظهرت مكتبات مثل CuPy وDask وJAX لهذا الغرض.
- المصفوفات متجانسة النوع؛ إذا احتجت لتخزين بيانات مختلطة الأنواع (نصوص وأرقام وتواريخ في جدول واحد) فإن Pandas أنسب.
- رسائل الخطأ المتعلقة بـ Broadcasting وأشكال المصفوفات (Shape Mismatch) قد تكون غير واضحة للمبتدئين.
- لا توفر بنية بيانات جدولية (Tabular) بأسماء أعمدة أو فهارس نصية؛ هذه مهمة Pandas.
- العمليات على مصفوفات ضخمة جدًا لا تتسع في الذاكرة (Out-of-core) تحتاج أدوات إضافية مثل Dask.

١.٧ متى تستخدمها ومتى لا تستخدمها

القرار	الحالة
استخدم NumPy مباشرة	بيانات عددية متجانسة (أرقام فقط) وحسابات رياضية/إحصائية
استخدم Pandas (المبنية فوق NumPy)	بيانات جدولية بأعمدة مسمّاة وأنواع مختلطة
استخدم PyTorch أو TensorFlow أو JAX	حسابات على GPU لتدريب نماذج تعلّم عميق ضخمة
استخدم Dask array	بيانات أكبر من ذاكرة الجهاز (Out-of-core)
استخدم NumPy مباشرة	عمليات مصفوفية بسيطة إلى متوسطة الحجم في السكربتات اليومية

١.٨ أشهر البدائل

- CuPy: نفس واجهة NumPy تقريبًا لكن التنفيذ على كروت الرسوميات (GPU).
- JAX: واجهة شبيهة بـ NumPy مع دعم للاشتقاق التلقائي (Autograd) والتجميع (JIT) عبر XLA.
- Dask Array: يقسّم مصفوفات NumPy الضخمة على أجزاء (Chunks) ويوزّع الحساب.
- PyTorch / TensorFlow Tensors: بنى بيانات مشابهة تدعم أيضًا التدرّج التلقائي والتدريب على GPU.

ملخص الفصل

- NumPy وُلدت لحل مشكلة بطء بايثون في الحوسبة العددية عبر مصفوفات متجانسة مُخزَّنة بشكل متصل في الذاكرة ومُعالَجة عبر شيفرة C.
- هي الأساس الذي تُبنى عليه تقريبًا كل مكتبات علم البيانات في بايثون.
- قوتها في الأداء والـ Vectorization، وحدودها في عدم دعم GPU أصلاً وعدم ملاءمتها للبيانات الجدولية المختلطة الأنواع.

الأسئلة الشائعة

س: هل أحتاج تعلّم NumPy إذا كنت سأستخدم Pandas فقط؟

ج: نعم، لأن Pandas مبنية فوق NumPy وتستخدم نفس مفاهيم المصفوفات والـ Broadcasting داخليًا، وفهمها يسهّل تشخيص الأخطاء وتحسين الأداء في Pandas.

س: هل NumPy بديل عن الرياضيات في تعلّم الآلي؟

ج: لا، هي أداة تنفيذ فقط؛ الرياضيات والخوارزميات خارج نطاق هذا الكتاب بشكل مقصود.

الفصل الثاني: التثبيت والإعداد

٢.١ طرق التثبيت

أشهر طريقتين لتثبيت NumPy هما pip وconda. اختيار الأداة المناسبة يعتمد على بيئة العمل: pip هو مدير الحزم الرسمي لبايثون ويعمل مع أي بيئة افتراضية عادية، بينما conda هو مدير حزم وبيئات معًا، وهو مفضّل في البيئات العلمية لأنه يدير أيضًا التبعيات غير البايثونية (مثل مكتبات BLAS/LAPACK المُجمَّعة التي تسرّع العمليات الخطية).

```
# باستخدام pip
pip install numpy

# باستخدام conda
conda install numpy

# تحديد إصدار معيّن
pip install numpy==1.26.4
```

٢.٢ البيئات الافتراضية

من أفضل الممارسات ألا تُثبّت أي مكتبة في بيئة بايثون النظام (System Python) مباشرة، بل داخل بيئة افتراضية معزولة لكل مشروع. هذا يمنع تعارض الإصدارات بين المشاريع المختلفة، ويجعل مشروعك قابلاً لإعادة الإنتاج (Reproducible) على أي جهاز آخر.

```
# إنشاء بيئة افتراضية باستخدام venv
python -m venv .venv
source .venv/bin/activate      # على Linux/Mac
.venv\Scripts\activate         # على Windows
pip install numpy

# باستخدام conda
conda create -n ds-env python=3.11 numpy pandas
conda activate ds-env
```

٢.٣ إدارة الإصدارات والتوافق

NumPy تصدر إصدارات رئيسية قد تحمل تغييرات كاسرة (Breaking Changes)، مثل الانتقال من NumPy 1.x إلى NumPy 2.0 الذي غيّر بعض السلوكيات الافتراضية (مثل قواعد الـ Type Promotion). لذلك من الضروري تثبيت الإصدارات في ملف متطلبات (requirements.txt) أو ملف بيئة (environment.yml) بدلاً من الاعتماد على أحدث إصدار دائمًا.

```
# requirements.txt
numpy>=1.26,<2.0

# التحقق من الإصدار المثبَّت
python -c "import numpy; print(numpy.__version__)"
```

٢.٤ أفضل أسلوب للاستيراد

المعيار المتعارف عليه عالميًا في كل التوثيق الرسمي والمشاريع مفتوحة المصدر هو استيراد NumPy باسم مستعار np. الالتزام بهذا المعيار يجعل الكود مقروءًا فورًا لأي مبرمج آخر.

```
import numpy as np

# تجنّب:
from numpy import *   # يلوّث الفضاء الاسمي ويخفي مصدر الدوال
```

أفضل الممارسات

- استخدم دائمًا import numpy as np ولا تغيّر هذا الاسم المستعار أبدًا.
- ثبّت مكتباتك داخل بيئة افتراضية معزولة لكل مشروع.
- ثبّت إصدارات محددة في requirements.txt لضمان قابلية إعادة الإنتاج.
- تحقّق من ملاحظات الإصدار (Release Notes) قبل الترقية إلى إصدار رئيسي جديد مثل NumPy 2.0.

٢.٥ تنظيم المشروع والملفات

في المشاريع الاحترافية، يُفصَل كود المعالجة العددية عن كود تحميل البيانات وعن كود العرض المرئي، عادة داخل حزمة بايثون منظمة بمجلدات واضحة.

```
project/
├── data/
│   ├── raw/
│   └── processed/
├── src/
│   ├── __init__.py
│   ├── preprocessing.py   # عمليات NumPy/Pandas
│   └── viz.py             # Matplotlib/Plotly
├── notebooks/
│   └── exploration.ipynb
├── requirements.txt
└── README.md
```

ملخص الفصل

- استخدم بيئات افتراضية معزولة (venv أو conda) لكل مشروع.
- ثبّت الإصدارات بدقة في requirements.txt لتجنّب مشاكل التوافق بين NumPy 1.x وNumPy 2.x.
- المعيار العالمي للاستيراد هو import numpy as np.

الفصل الثالث: فلسفة التصميم (Design Philosophy)

٣.١ الفكرة الجوهرية: مصفوفة واحدة متجانسة

يقوم تصميم NumPy بالكامل حول فكرة واحدة مركزية: ndarray، وهي كتلة بيانات متجانسة (كل عناصرها من نفس النوع) ذات شكل (Shape) محدد، مخزَّنة في مساحة ذاكرة متصلة (Contiguous Memory Block). هذا القرار التصميمي البسيط ظاهريًا هو ما يجعل كل العمليات سريعة وقابلة للتنبؤ، لأن المعالج يمكنه قراءة البيانات من الذاكرة بأنماط وصول منتظمة ومتوقعة (Cache-friendly access patterns).

بالمقارنة، قائمة بايثون list هي في الحقيقة مصفوفة من مؤشرات (Pointers) تشير إلى كائنات بايثون منتشرة في أماكن مختلفة من الذاكرة، وكل كائن رقمي في بايثون يحمل معه معلومات إضافية (نوعه، عدد مرات الإشارة إليه Reference Count، إلخ). هذا الغلاف الإضافي هو ما يجعل القوائم أبطأ وأقل كفاءة في استهلاك الذاكرة للبيانات العددية الكبيرة.

٣.٢ فصل الوصف عن البيانات: Metadata وBuffer

كل ndarray يتكوّن مفهوميًا من جزأين: كتلة البيانات الخام (Data Buffer)، ووصف بيانات وصفية (Metadata) يشمل الشكل (Shape)، ونوع البيانات (dtype)، والخطوات (Strides). هذا الفصل هو ما يجعل عمليات مثل reshape أو transpose سريعة جدًا: فهي لا تحرّك أي بايت واحد من البيانات الفعلية، بل تغيّر فقط البيانات الوصفية التي تصف كيفية قراءة نفس الذاكرة.

العنصر الوصفي	الدور
shape	عدد العناصر في كل بُعد، مثل (٣, ٤) لمصفوفة ٣ صفوف و٤ أعمدة.
dtype	نوع كل عنصر في المصفوفة (bool، float32، int64...).
strides	عدد البايتات التي يجب القفز بها في الذاكرة للانتقال بين عناصر كل بُعد.
ndim	عدد الأبعاد (١ لمتجه، ٢ لمصفوفة، إلخ).

٣.٣ Vectorization كفلسفة لا كميزة إضافية

الفلسفة الأعمق في NumPy ليست فقط 'مصفوفة سريعة'، بل تغيير طريقة تفكيرك في كتابة الكود العددي بالكامل: بدلاً من التفكير بعنصر واحد في كل مرة عبر حلقة for، يدفعك NumPy للتفكير بالمصفوفة ككل ككائن رياضي واحد تُطبَّق عليه العمليات دفعة واحدة. هذا النمط من التفكير يسمى Array-Oriented Programming، وهو متأثر مباشرة بلغات مثل APL وMATLAB.

```
# النمط غير الموصى به (تفكير عنصر بعنصر)
result = []
for x in data:
    result.append(x * 2 + 1)

# النمط النمطي في NumPy (تفكير بالمصفوفة ككل)
result = data * 2 + 1
```

٣.٤ Broadcasting: تعميم العمليات بدون نسخ

Broadcasting هو مجموعة قواعد تسمح لـ NumPy بتنفيذ عمليات بين مصفوفات ذات أشكال مختلفة دون الحاجة لنسخ البيانات فعليًا لتوسيعها. الفلسفة هنا هي: التوسّع منطقي (Logical) وليس فيزيائيًا (Physical) في الذاكرة، مما يوفّر ذاكرة وزمن معالجة.

```
a = np.array([1, 2, 3])        # shape (3,)
b = 10
a + b                          # array([11, 12, 13]) — b يُبَثّ منطقيًا لكل عنصر
```

٣.٥ البنية الداخلية: Modules وNamespaces

تنقسم NumPy داخليًا إلى وحدات فرعية (Submodules) لكل منها مسؤولية محددة، وهو انعكاس لفلسفة الفصل بين الاهتمامات (Separation of Concerns).

المسؤولية	الوحدة الفرعية
التطبيق الأساسي لـ ndarray وعمليات C منخفضة المستوى.	numpy.core / numpy._core
الجبر الخطي: محدّدات، قيم ذاتية، حل أنظمة معادلات.	numpy.linalg
توليد أرقام عشوائية وتوزيعات إحصائية.	numpy.random
تحويل فورييه السريع.	numpy.fft
المصفوفات المقنَّعة (Masked Arrays) للتعامل مع القيم المفقودة.	numpy.ma
التعامل مع كثيرات الحدود.	numpy.polynomial

٣.٦ فلسفة التوافق العكسي وسياسة NEP

يتّبع مطوّرو NumPy عملية رسمية لاقتراح التغييرات الكبرى تسمى NEP (NumPy Enhancement Proposal)، وهي مستوحاة من PEP الخاصة بلغة بايثون نفسها. هذا يعكس فلسفة تصميم تحترم الاستقرار طويل المدى، حيث لا تُدخَل تغييرات جذرية إلا بعد نقاش مجتمعي موثَّق، وهو ما يفسّر لماذا استغرق الانتقال إلى NumPy 2.0 سنوات من التخطيط.

ملخص الفصل

- التصميم بالكامل يدور حول ndarray: بيانات متجانسة متصلة في الذاكرة، مع بيانات وصفية منفصلة (shape, dtype, strides).
- الفلسفة المركزية هي التفكير بالمصفوفة ككل (Vectorization) بدلاً من التفكير بعنصر واحد في كل مرة.
- Broadcasting يسمح بعمليات بين أشكال مختلفة دون نسخ فيزيائي للبيانات.
- البنية الداخلية مقسّمة لوحدات فرعية واضحة المسؤولية (linalg, random, fft, ma...).

الفصل الرابع: المفاهيم الأساسية (Core Concepts)

٤.١ ndarray: الكائن المركزي

ndarray هو الكائن المركزي في NumPy، ويمكن إنشاؤه بعدة طرق: من قائمة بايثون، أو عبر دوال بناء جاهزة، أو بقراءة بيانات من ملف.

```
import numpy as np

a = np.array([1, 2, 3])              # من قائمة
b = np.zeros((2, 3))                 # مصفوفة أصفار بشكل (2, 3)
c = np.ones((3, 3))                  # مصفوفة آحاد
d = np.arange(0, 10, 2)              # [0 2 4 6 8]
e = np.linspace(0, 1, 5)             # 5 قيم متساوية التباعد بين 0 و1
f = np.eye(3)                        # مصفوفة الوحدة 3×3
```

٤.٢ الأبعاد والشكل (Dimensions & Shape)

ndim يمثّل عدد الأبعاد، وshape هو Tuple يصف طول كل بُعد. المتجه أحادي البُعد (١D) يختلف مفهوميًا عن المصفوفة ثنائية البُعد ذات صف واحد أو عمود واحد؛ هذا الفرق مصدر كثير من أخطاء المبتدئين.

```
v = np.array([1, 2, 3])          # shape (3,)   -> 1 متجهD
row = np.array([[1, 2, 3]])      # shape (1, 3) -> 2 مصفوفةD بصف واحد
col = np.array([[1], [2], [3]])  # shape (3, 1) -> 2 مصفوفةD بعمود واحد

print(v.ndim, row.ndim, col.ndim)   # 1 2 2
```

٤.٣ Stride: كيف تُقرأ الذاكرة فعليًا

الـ Strides توضّح عدد البايتات التي يجب القفز بها في الذاكرة الخام للانتقال بين عنصرين متتاليين على كل بُعد. فهم Strides يفسّر لماذا بعض العمليات (كإعادة التشكيل reshape) تكاد لا تستهلك وقتًا، بينما عمليات أخرى (كـ transpose ثم تعديل عنصر) قد تتطلب انتباهًا لأنها لا تنسخ البيانات.

```
a = np.arange(12).reshape(3, 4)
print(a.strides)   # (32, 8) 64 على نظام bit- مع dtype=int64
# 4( عناصر  8× بايت): قفزة 32 بايت للانتقال صفًا واحدًا
# (عنصر واحد): قفزة 8 بايت للانتقال عمودًا واحدًا
```

٤.٤ Views مقابل Copies

هذا من أهم المفاهيم في NumPy على الإطلاق. الـ View هو مصفوفة جديدة تشير إلى نفس الذاكرة الأصلية (لا نسخ بيانات)، بينما الـ Copy ينشئ كتلة ذاكرة جديدة تمامًا مستقلة عن الأصل. تعديل عنصر في View يُغيّر المصفوفة الأصلية أيضًا، بينما تعديل عنصر في Copy لا يؤثر على الأصل إطلاقًا.

```
a = np.arange(6)
b = a[1:4]        # View — تشير لنفس الذاكرة (Slicing العادي ينشئ Views)
b[0] = 99
print(a)          # [ 0 99  2  3  4  5] — تأثّرت المصفوفة الأصلية!
```

```
c = a[1:4].copy() # Copy صريحة ومستقلة
c[0] = -1
print(a)          # لم تتأثر
```

القاعدة العامة: التقطيع الأساسي (Basic Slicing) بأرقام وفواصل (:) ينتج Views، بينما الفهرسة المتقدمة (Fancy Indexing) بقوائم أو مصفوفات من الفهارس، والفهرسة المنطقية (Boolean Indexing)، تنتج دائمًا Copies.

٤.٥ Broadcasting بالتفصيل

قواعد Broadcasting الرسمية: تتم مقارنة أشكال المصفوفتين بدءًا من آخر بُعد. بُعدان متوافقان إذا كانا متساويين، أو كان أحدهما يساوي ١. إذا كان عدد الأبعاد مختلفًا، تُضاف أبعاد بقيمة ١ من جهة اليسار تلقائيًا للمصفوفة الأقصر.

الشكل الأول	الشكل الثاني	النتيجة
(٤ ,٣)	(,٤)	متوافق -> (٤ ,٣)
(١ ,٣)	(٤ ,١)	متوافق -> (٤ ,٣)
(٤ ,٣)	(,٣)	غير متوافق -> خطأ ValueError
(٣ ,١ ,٥)	(٣ ,٤)	متوافق -> (٥, ٤ ,٣)

```
a = np.ones((3, 4))
b = np.array([1, 2, 3, 4])   # shape (4,)
a + b                        # b يُبَثّ ليضاف لكل صف من a
```

٤.٦ الفهرسة والتقطيع (Indexing & Slicing)

الفهرسة الأساسية

```
a = np.arange(10)
a[2]        # 2
a[2:5]      # [2 3 4]
a[::-1]     # عكس المصفوفة

m = np.arange(12).reshape(3, 4)
m[1, 2]     # العنصر في الصف 1 والعمود 2
m[:, 0]     # كل العناصر في العمود الأول
m[0, :]     # كل عناصر الصف الأول
```

الفهرسة المتقدمة (Fancy Indexing)

```
a = np.array([10, 20, 30, 40, 50])
idx = [0, 2, 4]
a[idx]          # [10 30 50] — نسخة جديدة وليست View
```

الفهرسة المنطقية (Boolean Indexing)

```
a = np.array([1, -2, 3, -4, 5])
mask = a > 0
a[mask]          # [1 3 5]
a[a > 0] = 0     # تصفير كل القيم الموجبة في المصفوفة الأصلية
```

٤.٧ إعادة التشكيل والنقل والتفريد

```
a = np.arange(12)
b = a.reshape(3, 4)      # نفس البيانات، شكل جديد (غالبًا View)
c = b.T                  # Transpose — تبديل الصفوف بالأعمدة (View)
d = b.flatten()          # 1D تحويل لمتجه — دائمًا Copy
e = b.ravel()            # 1D تحويل لمتجه — View إن أمكن، وإلا Copy
```

الفرق بين flatten وravel مثال ممتاز على فلسفة NumPy في إعطاء المبرمج تحكمًا دقيقًا: flatten تضمن دائمًا نسخة مستقلة (أكثر أمانًا لكن أبطأ قليلاً وأكثر استهلاكًا للذاكرة)، بينما ravel تحاول تجنّب النسخ قدر الإمكان (أسرع لكن قد تُرجع View فتؤثر تعديلاتها على الأصل).

الأخطاء الشائعة

- الخلط بين View وCopy وتعديل مصفوفة أصلية بالخطأ عبر Slicing.
- افتراض أن shape=(3,) و shape=(3,1) متماثلان في العمليات، بينما Broadcasting يعاملهما بشكل مختلف تمامًا.
- استخدام حلقات for بايثون بدلاً من العمليات المتجهة (Vectorized)، مما يهدر ميزة الأداء الأساسية في NumPy.

تمارين عملية

١. أنشئ مصفوفة 2D بشكل (٤, ٥) تحتوي أرقامًا من ٠ إلى ١٩ باستخدام arange وreshape، ثم اطبع صفها الأخير وعمودها الأول.

٢. بيّن عمليًا الفرق بين View وCopy عبر إنشاء تقطيع من مصفوفة وتعديل عنصر فيه، ثم افحص هل تأثرت المصفوفة الأصلية.

٣. استخدم Boolean Indexing لاستخراج كل القيم الزوجية من مصفوفة عشوائية من ٢٠ عنصرًا.

حلول مقترحة (تلميحات)

١. استخدم np.arange(20).reshape(4, 5) ثم [-١, :] و[:, ٠].

٢. قارن معرّف الذاكرة عبر np.shares_memory(a, b) قبل الحكم.

٣. استخدم الشرط a[a % 2 == 0].

الفصل الخامس: الكائنات وواجهات البرمجة (Objects & APIs)

٥.١ الكائن ndarray: الخصائص الأساسية

الخاصية (Attribute)	الوصف
shape	شكل المصفوفة كـ Tuple.
dtype	نوع بيانات العناصر.
ndim	عدد الأبعاد.
size	إجمالي عدد العناصر.
itemsize	حجم كل عنصر بالبايت.
nbytes	إجمالي حجم المصفوفة بالبايت (size × itemsize).
T	المصفوفة المنقولة (Transpose) كـ View.

٥.٢ أنواع البيانات (dtype)

تحديد dtype بدقة له أثر مباشر على الأداء واستهلاك الذاكرة. استخدام float64 حيثما يكفي float32 يضاعف استهلاك الذاكرة بلا داعٍ، وهو خطأ شائع في المشاريع الكبيرة.

dtype	الوصف	الحجم
int32 / int64	أعداد صحيحة	٤ / ٨ بايت
float32 / float64	أعداد عشرية	٤ / ٨ بايت
bool	قيم منطقية	١ بايت
object	كائنات بايثون عامة (يفقد مزايا NumPy)	متغيّر

```
a = np.array([1, 2, 3], dtype=np.float32)
a.astype(np.int64)   # تحويل النوع – دائمًا ينتج Copy
```

٥.٣ الدوال مقابل الطرق (Functions vs Methods)

في NumPy غالبًا ما تجد نفس العملية مُتاحة كدالة مستقلة وكطريقة (Method) على الكائن نفسه، وهذا خيار تصميمي متعمَّد يمنح مرونة في الأسلوب البرمجي، خاصة مع Method Chaining.

```
np.sum(a)       # كدالة مستقلة
a.sum()         # كطريقة على الكائن – الأسلوب الأكثر شيوعًا في الكود الاحترافي

# Method Chaining: تسلسل عمليات في سطر واحد مقروء
result = a.reshape(3, 4).sum(axis=1).round(2)
```

٥.٤ نموذج التنفيذ: Eager Execution

NumPy تتبع نموذج تنفيذ فوري (Eager Execution): كل عملية تُنفَّذ فور كتابتها وتُخصَّص لها الذاكرة مباشرة، بعكس بعض أطر التعلّم العميق التي تبني رسمًا حسابيًا (Computational Graph) أولاً ثم تنفّذه لاحقًا (Lazy

Evaluation). هذا يجعل NumPy أسهل تصحيحًا (Debugging) لأن كل سطر ينتج قيمة فعلية فورية يمكن طباعتها وفحصها.

٥.٥ Vectorization كنموذج تنفيذ

عندما تكتب a + b لمصفوفتين، فإن NumPy لا يستدعي دالة جمع بايثون لكل عنصر، بل يستدعي حلقة C واحدة مُجمَّعة مسبقًا (تسمى Universal Function أو ufunc) تعالج كل العناصر دفعة واحدة. هذا هو مصدر السرعة الحقيقي، وليس مجرد "اختصار في الكتابة".

```
np.add(a, b)      # ufunc تُنفَّذ في C
a + b             # نفس الـ ufunc عبر Operator Overloading
```

٥.٦ نموذج الذاكرة (Memory Model)

كل ndarray يشير إلى كتلة ذاكرة واحدة (Buffer)، وقد يتشارك أكثر من كائن ndarray نفس الـ Buffer (كما في حالة Views). لهذا يوفّر NumPy دالة np.shares_memory() للتحقق مما إذا كانت مصفوفتان تتشاركان نفس الذاكرة الفعلية، وهي أداة تشخيصية مهمة عند تتبّع أخطاء التعديل غير المقصود.

```
a = np.arange(10)
b = a[2:5]
np.shares_memory(a, b)   # True
```

أفضل الممارسات

- فضّل صيغة الطريقة a.sum() على الدالة المستقلة np.sum(a) في الكود اليومي، فهي أكثر تناسقًا مع أسلوب Method Chaining.
- حدّد dtype صراحة عند إنشاء مصفوفات كبيرة لتفادي هدر الذاكرة.
- استخدم np.shares_memory() عند الشك في وجود مشاركة ذاكرة غير مقصودة بين مصفوفتين.

الفصل السادس: أهم الدوال (Core Functions)

هذا الفصل يغطي أهم الدوال التي ستستخدمها يوميًا في NumPy، مصنّفة حسب الغرض، مع الصياغة (Syntax) والمعاملات والقيمة المُرجَعة وأفضل الممارسات والأخطاء الشائعة لكل مجموعة.

٦.١ دوال إنشاء المصفوفات

الدالة	الوصف	مثال
np.array(obj, dtype=None)	تحويل قائمة/تسلسل إلى ndarray	np.array([1,2,3])
np.zeros(shape)	مصفوفة أصفار	np.zeros((2,3))
np.ones(shape)	مصفوفة آحاد	np.ones((2,3))
np.full(shape, val)	مصفوفة بقيمة ثابتة	np.full((2,2), 7)
np.arange(start,stop,step)	متتالية بخطوة ثابتة	np.arange(0,10,2)
np.linspace(start,stop,num)	عدد ثابت من القيم متساوية التباعد	np.linspace(0,1,5)
np.eye(n)	مصفوفة الوحدة	np.eye(3)
np.random.rand(*shape)	قيم عشوائية منتظمة [٠،١)	np.random.rand(2,2)

ملاحظة أداء: np.zeros و np.empty كلاهما يخصّص ذاكرة بنفس الشكل، لكن np.empty لا يهيّئ القيم (تبقى قيمًا عشوائية من الذاكرة القديمة)، لذا فهي أسرع قليلاً لكنها خطرة إن نسيت ملء المصفوفة قبل استخدامها.

٦.٢ العمليات الحسابية والإحصائية

الدالة	Syntax	الوصف
np.sum	a.sum(axis=None)	مجموع العناصر، مع إمكانية التحديد حسب المحور
np.mean	a.mean(axis=None)	المتوسط الحسابي
np.std / np.var	()a.std(), a.var	الانحراف المعياري والتباين
np.min / np.max	()a.min(), a.max	أصغر وأكبر قيمة
np.argmin / np.argmax	()a.argmin	فهرس أصغر/أكبر قيمة
np.round	np.round(a, decimals)	تقريب العناصر
np.cumsum	()a.cumsum	المجموع التراكمي

معامل axis من أهم المفاهيم هنا: axis=0 يعني 'اجمع عبر الصفوف' (أي لكل عمود)، و axis=1 يعني 'اجمع عبر الأعمدة' (أي لكل صف). هذا عكس ما يتوقعه كثير من المبتدئين بديهيًا.

```
m = np.arange(6).reshape(2, 3)
m.sum()          # 15 — مجموع كل العناصر
m.sum(axis=0)    # [3 5 7]  — مجموع كل عمود عبر الصفوف
m.sum(axis=1)    # [3 12]   — مجموع كل صف عبر الأعمدة
```

٦.٣ الجبر الخطي (numpy.linalg)

الدالة	الوصف
np.dot(a, b) / a @ b	الضرب المصفوفي (Matrix Multiplication)
np.linalg.inv(a)	معكوس المصفوفة
np.linalg.det(a)	المحدّد (Determinant)
np.linalg.eig(a)	القيم والمتجهات الذاتية
np.linalg.solve(a, b)	حل نظام معادلات خطية Ax = b

تنبيه شائع: عامل الضرب * بين مصفوفتين ٢D لا يعني الضرب المصفوفي الرياضي، بل ضرب كل عنصر بمقابله (Element-wise). للضرب المصفوفي الحقيقي استخدم @ أو np.dot أو np.matmul.

```
A = np.array([[1, 2], [3, 4]])
B = np.array([[5, 6], [7, 8]])
A * B       # ضرب عنصر بعنصر – ليس ضربًا مصفوفيًا
A @ B       # الضرب المصفوفي الصحيح
```

٦.٤ الوحدة العشوائية (numpy.random)

منذ NumPy 1.17، أصبح الأسلوب الموصى به هو استخدام Generator عبر np.random.default_rng() بدلاً من الدوال القديمة (Legacy) مثل np.random.seed()، لأن Generator يوفّر عزلًا أفضل للحالة العشوائية (State) بين أجزاء الكود المختلفة.

```
# الأسلوب الحديث الموصى به
rng = np.random.default_rng(seed=42)
rng.random(5)           # 5 أرقام عشوائية منتظمة
rng.integers(0, 10, 5) # 5 أعداد صحيحة عشوائية
rng.normal(0, 1, 5)    # 1 توزيع طبيعي بمتوسط 0 وانحراف

# الأسلوب القديم (ما زال شائعًا لكنه غير مفضّل في الكود الجديد)
np.random.seed(42)
np.random.rand(5)
```

٦.٥ ملخص Edge Cases شائعة

الحالة	السلوك
القسمة على صفر float	لا يرفع استثناء، بل يُرجع inf أو nan مع تحذير (RuntimeWarning)
مقارنة مصفوفتين بـ ==	تُرجع مصفوفة منطقية عنصر بعنصر، وليست قيمة واحدة True/False
استخدام مصفوفة NumPy في شرط if مباشرة	يرفع ValueError إن كانت المصفوفة تحتوي أكثر من عنصر
الجمع بين int وfloat	يتم رفع النوع تلقائيًا (Type Promotion) إلى float

```
a = np.array([1, 2, 3])
b = np.array([1, 2, 4])
a == b            # array([ True,  True, False])
```

```
if a == b:          # يرفع ValueError: عدة قيم غامضة
    pass
# البديل الصحيح:
if np.array_equal(a, b):
    pass
```

الأخطاء الشائعة

- استخدام if array مباشرة بدل np.all() أو np.any() أو np.array_equal().
- الخلط بين * (ضرب عنصر بعنصر) و@ (ضرب مصفوفي).
- نسيان أن np.empty لا يصفّر الذاكرة.
- الاعتماد على np.random.seed العام بدلاً من Generator معزول في كود يُستخدم كمكتبة.

الأسئلة الشائعة

س: ما الفرق بين np.dot وnp.matmul؟

ج: لمصفوفات ٢D النتيجة متطابقة عادة، لكن matmul (وعامل @) هو المعيار الحديث ويتعامل بشكل أوضح مع الأبعاد الأعلى (Batched matrix multiplication)، بينما dot له قواعد أقدم وأقل وضوحًا في تلك الحالات.

س: متى أستخدم float32 بدل float64؟

ج: عند التعامل مع بيانات ضخمة جدًا أو عند التغذية لنماذج تعلّم عميق تتوقع float32، حيث يقلّل استهلاك الذاكرة إلى النصف تقريبًا مقابل فقدان بسيط في الدقة.

تمارين عملية

١. أنشئ مصفوفتين ٣×٣ واحسب لهما: الجمع العنصري، الضرب العنصري، والضرب المصفوفي، ووضّح الفرق بين النتيجتين الأخيرتين.

٢. استخدم axis=0 وaxis=1 لحساب متوسط كل عمود ومتوسط كل صف في مصفوفة ٤×٣.

٣. ولّد ١٠٠٠ رقم عشوائي من توزيع طبيعي باستخدام Generator الحديث، واحسب متوسطها وانحرافها المعياري.

حلول مقترحة (تلميحات)

١. استخدم + للجمع، * للضرب العنصري، و@ أو np.dot للضرب المصفوفي، ولاحظ اختلاف الأشكال والقيم.

٢. m.mean(axis=0) للأعمدة، m.mean(axis=1) للصفوف.

٣. rng = np.random.default_rng(); data = rng.normal(0, 1, 1000); data.mean(); data.std().

مراجع وروابط رسمية

- التوثيق الرسمي: https://numpy.org/doc/stable/
- دليل المستخدم (User Guide): https://numpy.org/doc/stable/user/index.html

- مرجع واجهة البرمجة (API Reference): https://numpy.org/doc/stable/reference/index.html
- الكود المصدري على GitHub: https://github.com/numpy/numpy
- ورقة NumPy البحثية في Nature (2020): Harris, C.R., et al. Array programming with NumPy.

خاتمة الجزء الأول

بهذا نكون قد أرسينا الأساس النظري والعملي لفهم NumPy: تاريخها وفلسفتها وبنيتها الداخلية ومفاهيمها الجوهرية (Broadcasting، Views، Strides، Shape)، إضافة إلى أهم دوالها. الجزء الثاني سيبني على هذا الأساس عبر أمثلة عملية متدرجة من مشاريع حقيقية، ثم يتعمّق في الأداء (Performance) والتصحيح (Debugging) وأفضل الممارسات الاحترافية، وينتهي بمشاريع مصغّرة تطبيقية.